

TESTE DE SOFTWARE

Disciplina: Engenharia de Requisitos, Teste e Qualidade de Software

Aluno: Rhwan Guilherme Ferreira Torres

Matricula: 01696766

● Conceito de teste de software

O teste de software é basicamente o processo de verificar se um sistema está funcionando corretamente ou não. Isso é feito executando o sistema com diferentes dados de entrada e analisando se o resultado que ele gera está de acordo com o esperado. De forma mais simples, testar é pegar o sistema, usar ele e observar se ele se comporta do jeito certo ou não.

Mas é importante entender que testar não é a mesma coisa que usar o sistema normalmente. Quando uma pessoa usa um sistema no dia a dia, ela só quer que funcione. Já quando alguém está testando, a ideia é justamente tentar encontrar erros. Muitas vezes, quem testa tenta usar o sistema de formas diferentes do comum, justamente pra ver se algo dá errado.

exemplo:

- Tentar fazer login sem senha
- Inserir dados inválidos em um cadastro

Outro coisa importante é que testar não significa apenas olhar o código e achar que está certo. Mesmo que o código pareça certo, na prática o sistema pode apresentar erros. Por isso o teste é considerado uma atividade dinâmica, ou seja, precisa rodar o sistema de verdade.

Ele também está ligado à qualidade do produto final. Quanto mais um sistema é testado, maiores são as chances de ele funcionar corretamente em diferentes situações. Isso faz com que o sistema seja mais confiável e tenha menos problemas durante o uso. Por isso, o teste não deve ser visto como algo opcional, mas sim como algo obrigatório ou uma parte essencial do desenvolvimento.

● Objetivos dos Testes

O objetivo dos testes de software vai muito além de simplesmente ver se o sistema funcionar. Na verdade, testar é uma das partes mais importantes do desenvolvimento, porque é através dos testes que a gente consegue identificar problemas, melhorar a qualidade do sistema e evitar que erros cheguem até o usuário final.

De forma mais simples, o principal objetivo dos testes é encontrar defeitos no sistema antes que ele seja usado pelas pessoas. Isso é importante porque, quando um erro chega até o usuário, o impacto pode ser muito maior. Pois dependendo do tipo de sistema, isso pode causar desde pequenos incômodos as pessoas, até problemas graves, como perda de dados, prejuízos financeiros ou falhas de segurança.

Por exemplo, um sistema de banco que calcula o saldo errado, ou um sistema de escola que perde informações dos alunos. Esses problemas poderiam ser evitados se fossem encontrados durante os testes. Por isso, testar é uma forma de prevenir problemas.

Outro objetivo muito importante dos testes é melhorar a qualidade do software. Um sistema que passa por vários testes tem mais chances de funcionar melhor, travar menos, ter menos erros e oferecer uma experiência melhor para o usuário. Isso faz com que as pessoas confiem mais no sistema criado.

Além disso, os testes ajudam a reduzir custos. Pois por mais que as muitas coisas sejam feitas, testar ajuda a economizar dinheiro. Isso acontece porque corrigir um erro durante o desenvolvimento é muito mais fácil e barato do que corrigir depois que o sistema já está sendo usado pelos usuários. Quando um erro é encontrado tarde demais, pode ser necessário refazer várias partes do sistema, o que dá muito mais trabalho.

Outro ponto importante é que os testes ajudam os desenvolvedores a trabalharem com mais segurança e conforto. Quando existem testes bem feitos, a equipe pode fazer mudanças no sistema sem tanto medo, porque sabem que os testes vão mostrar se algo parou de funcionar ou continua funcionando perfeitamente. Isso é muito comum em sistemas grandes, que estão sempre sendo atualizados.

Os testes também ajudam a entender melhor o sistema. Pois, muitas vezes durante o processo de testar, a gente descobre coisas que não estavam claras no começo, como regras mal definidas ou funcionalidades que não estão funcionando como deveriam.

Além disso, os testes ajudam a evitar falhas em produção, ou seja, quando o sistema já está sendo usado de verdade por pessoas ou clientes. Quanto menos erros chegarem até o usuário, melhor será a qualidade percebida do sistema. Isso é muito

importante para grandes ou pequenas empresas, porque um sistema com muitos erros pode prejudicar a imagem da empresa.

Outro objetivo dos testes é garantir que o sistema atende aos requisitos definidos. Ou seja, verificar se aquilo que foi pedido realmente foi entregue. Pois, as vezes o sistema funciona, mas não faz exatamente como o cliente queria.

Também é importante lembrar que os testes não servem para provar que o sistema está perfeito, porque na prática, não é possível testar todas as situações possíveis, principalmente em sistemas grandes. O principal objetivo dos testes é aumentar a confiança de que o sistema está funcionando bem.

Além disso, os testes também ajudam a identificar pontos fracos do sistema. Onde durante os testes, é possível perceber que algumas partes do sistema são mais problemáticas ou mais difíceis de usar, o que pode indicar que precisam ser melhoradas.

Em caso de atualizações constantes no sistema, os testes também ajudam a garantir que novas mudanças não quebrem funcionalidades antigas. Pois as vezes uma alteração pequena pode causar problemas em outras partes do sistema.

No geral, os testes têm vários objetivos:

- Encontrar erros antes do usuário
- Melhorar a qualidade do sistema
- Reduzir custos de correção
- Aumentar a confiança no software
- Ajudar os desenvolvedores
- Garantir que o sistema atende aos requisitos
- Evitar problemas em produção

Portanto, dá pra entender que testar não é só uma etapa do desenvolvimento, mas sim uma das partes mais importantes de todo o processo. Sem testes, o sistema pode até funcionar, mas sempre vai ter um risco muito maior de apresentar problemas.

Também vale destacar que os testes ajudam a melhorar a organização do desenvolvimento. Quando uma equipe utiliza testes com frequência, fica mais fácil identificar onde estão os problemas e como resolvê-los. Isso torna o processo mais eficiente e evita retrabalho, contribuindo para um desenvolvimento mais rápido e com mais qualidade.

● Diferença entre erro, defeito e falha

É um dos pontos mais importantes dentro de testes de software, mas também é um dos que mais confundem. Pois na área de testes de software, é muito importante entender a diferença entre erro, defeito e falha. porque no dia a dia a gente chama tudo de erro. Mas na área de software cada coisa tem um significado.

O defeito: É o problema que está no código ou na lógica do sistema. Ele pode ter sido causado por um erro humano, como interpretar um requisito de forma errada, escrever um código incorreto ou esquecer alguma validação.

O defeito existe mesmo que o sistema não esteja sendo executado. Pois se não corretamente identificado Ele vai estar lá, no código.

Por exemplo:

- O programador escreveu uma fórmula errada
- Interpretou o requisito de forma errada
- Esqueceu de validar um campo
- Colocou uma condição errada (tipo > ao invés de <)
- Um sistema deveria calcular desconto de 10%, mas o código foi feito pra calcular 5%.

O defeito é o tipo de problema mais difícil de perceber sem testes, porque ele não aparece sozinho. Ele precisa ser “ativado” quando o sistema é iniciado pelo desenvolvedor.

O erro: Acontece durante a execução do sistema. Quando o sistema roda e entra em um estado incorreto por causa de um defeito, ou seja, o erro só aparece quando o sistema é executado. E nem sempre o erro é visível para os usuários, pois as vezes o sistema está com um comportamento interno errado, mas ainda não deu problema visível.

Exemplo:

- O sistema faz um cálculo errado internamente.
- Salva um valor errado no banco de dados.
- Lógica executando um caminho errado.
- Ignora uma validação.

O erro é basicamente o momento em que o defeito começa a “agir”.

A falha: é quando o problema aparece de forma visível. É quando o sistema apresenta um comportamento diferente do esperado, como travar, mostrar um valor errado ou permitir uma ação incorreta.

Exemplos:

- Sistema trava
- Aparece um valor errado

- Login funciona sem senha
- Botão não responde
- Página não carrega
- Sistema fecha sozinho

A falha é o que realmente atinge diretamente o usuário. Por isso, na prática quando alguém reclama de um sistema, geralmente está falando de uma falha.

Exemplos gerais:

Um sistema que calcula desconto:

- Defeito: fórmula errada no código
- Erro: cálculo incorreto durante execução
- Falha: valor errado mostrado para o usuário

Sistema de Login:

- Defeito: validação de senha incorreta
- Erro: sistema não verifica senha corretamente
- Falha: usuário entra sem senha

Nem todo defeito gera uma falha instantaneamente. As vezes o defeito só aparece em situações específicas. Por isso, os testes são importantes para verificar diferentes cenários e situações.

● Níveis de teste (unitário, integração, sistema, aceitação)

Os níveis de teste existem pra organizar melhor como o sistema vai sendo testado ao longo do desenvolvimento. Em vez de simplesmente testar tudo de uma vez, a gente divide o processo em etapas. Isso ajuda muito, porque fica mais fácil identificar onde está o problema quando algo dá errado.

Basicamente, os testes começam nas partes menores do sistema e vão crescendo até chegar no sistema completo. É como se fosse em uma construção: primeiro testa os blocos pequenos, depois junta tudo e testa o conjunto.

Essa divisão é importante porque permite encontrar erros mais cedo. E quanto mais cedo um erro é encontrado, mais fácil e barato é corrigir e melhorar.

Teste Unitário:

O teste unitário é o nível mais básico de todos. Ele testa pequenas partes do sistema, como funções, métodos ou partes específicas do código.

Geralmente, quem faz esse tipo de teste é o próprio programador, porque ele sabe exatamente o que aquela parte deveria fazer.

Por exemplo:

Se eu tenho uma função que soma dois números, eu posso testar assim:

$2 + 2 = 4$

$10 + 5 = 15$

$0 + 3 = 3$

Se algum resultado estiver errado, já dá pra saber que o problema está naquela função.

Esse tipo de teste é muito importante porque pega erros simples logo no começo, como:

- Cálculo errado
- Condição mal feita
- Lógica invertida
- Esquecimento de validação

Outra vantagem é que quando o sistema cresce, já tem várias partes testadas individualmente, o que facilita muito o desenvolvimento. Porém, ele tem um limite, pois o teste unitário não garante que o sistema inteiro funcione, por causa que ele testa tudo separado.

Teste de Integração:

Depois do teste das partes separadas, vem o teste de integração. Aqui a ideia é verificar se essas partes funcionam corretamente quando estão juntas. Isso é importante porque, muitas vezes uma função funciona sozinha, outra também funciona, mas quando se juntam, dá problema

Exemplo:

Um sistema de login pode ter

- uma função que verifica o usuário
- outra que verifica a senha

Separadas, funcionam. Mas juntas, pode acontecer de não se comunicarem direito, passar dados errados ou até falhar completamente

Outro exemplo bem comum é banco de dados: O sistema salva corretamente, mas na hora de buscar, não encontra

Ou:

envia dado errado pro banco

Esse tipo de teste pega problemas como:

- Erro na troca de informações
- Falha na comunicação entre partes
- Dados que não chegam corretamente
- Integração mal feita

Ou seja, ele faz com que o sistema funcione como um conjunto, e não só em partes isoladas.

Teste de Sistema:

O teste de sistema já é um nível mais completo. Aqui o sistema já está todo pronto, com todas as partes integradas. O objetivo agora é testar o sistema inteiro, como se fosse um usuário utilizando. Ou seja, a ideia é simular o uso real do sistema.

Exemplo:

- Entrar no sistema
- Fazer login
- Cadastrar um usuário
- Salvar
- Consultar depois

Tudo funcionando como um fluxo completo.

Nesse nível, são testadas:

- Funcionalidades completas
- Regras do sistema
- Fluxo de uso
- Comportamento geral

Além disso, também começam a ser analisados outros pontos importantes, como:

- Desempenho (se o sistema é rápido ou lento)
- Segurança (se tem falhas)
- Estabilidade (se trava ou não)

As vezes, tudo funciona nas partes menores, mas no sistema completo aparecem problemas que antes não eram visíveis.

Teste de Aceitação:

O teste de aceitação é o último nível e tem um foco diferente, que é verificar se o sistema atende o que o usuário realmente precisa. Ou seja, não é só ver se funciona, mas se faz sentido pro usuário.

Esse teste normalmente é feito por um cliente, um usuário final ou alguém representando eles.

Exemplo:

Um sistema pode estar funcionando, mas:

- É difícil de usar

- Não tem o que o usuário precisava
- Ser confuso e bagunçado

Nesse caso, ele pode falhar no teste de aceitação. Se passar nesse teste, significa que o sistema está pronto pra ser utilizado.

No Geral, os níveis consistem em:

1. Teste unitário

Testa função de salvar nome.

2. Teste de integração

Testa se o nome está sendo salvo no banco.

3. Teste de sistema

Testa cadastro completo funcionando.

4. Teste de aceitação

Verifica se o cadastro atende o que o usuário precisa.

● Tipos de teste (funcional e não funcional)

Além de dividir os testes em níveis, também dá pra classificar os testes em tipos. Os dois principais tipos são os testes funcionais e os não funcionais. Cada um deles analisa o sistema de uma forma diferente, e os dois são muito importantes.

Teste Funcional:

O teste funcional é aquele que verifica se o sistema está fazendo o que deveria fazer. Ou seja, ele analisa as funcionalidades do sistema, aquilo que o usuário realmente utiliza. Nesse tipo de teste, a gente não se preocupa com o código por trás, nem com como o sistema foi desenvolvido. O foco é só no comportamento.

Por isso, ele é conhecido como teste “caixa-preta”, porque a gente não olha o que tem dentro do sistema, só observa a entrada e a saída.

Exemplo:

Sistema de login:

entrada: usuário e senha

resultado esperado: acesso ao sistema

Se funcionar = ok

Se não funcionar = tem erro

Outros exemplos de teste funcional:

- Cadastro de usuário
- Busca de produtos
- Cálculo de valores
- Envio de formulário
- Emissão de relatórios

Ou seja, tudo que pode ser testado funcionalmente.

Apesar do teste funcional ser muito importante, ele não analisa tudo e também não verifica, por exemplo:

- Se o sistema é rápido
- Se é seguro
- Se trava
- Se aguenta muitos usuários

Ou seja, ele garante o funcionamento, mas não a qualidade do funcionamento.

Teste Não Funcional:

O teste não funcional é responsável por verificar como o sistema funciona. Ou seja, ele não está preocupado com as funcionalidades em si, mas com a qualidade do sistema. Ele analisa coisas como:

- Desempenho
- Segurança
- Usabilidade
- Estabilidade
- Confiabilidade

Teste de Desempenho:

Verifica se o sistema é rápido ou lento.

Exemplo:

- Sistema demora muito pra carregar
- Demora pra salvar os dados
- Fica lento com muitos usuários

Teste de Segurança:

Verifica se o sistema protege os dados corretamente.

Exemplo:

- Da pra entrar sem senha?
- Dados estão expostos?
- O sistema permite acesso indevido?

Teste de Usabilidade:

Verifica se o sistema é fácil de usar.

Exemplo:

- O usuário entende como usar?
- A interface é clara?
- É confuso ou simples?

Teste de Estabilidade:

Verifica se o sistema funciona sem travar.

Exemplo:

- sistema fecha sozinho
- trava com frequência
- Da erro inesperado

Os testes funcionais e não funcionais são essenciais para garantir a qualidade do sistema. O teste funcional garante que o sistema funciona corretamente, enquanto o teste não funcional garante que ele funcione bem, com qualidade. Os dois se complementam, e juntos ajudam a criar sistemas mais confiáveis, seguros e eficientes.

Concluindo

Testar é uma parte simples de conferir se o sistema funciona, mas na prática é muito mais do que isso. Os testes ajudam a garantir que tudo esteja funcionando direito antes de chegar no usuário.

Testar não é algo que deve ser feito só no final. O ideal é ir testando durante todo o desenvolvimento, porque assim fica mais fácil encontrar e corrigir erros. Se deixar pra testar tudo no final, pode acabar acumulando vários problemas e ficando muito mais difícil de resolver.

No geral, dá pra concluir que os testes de software são uma parte muito importante do desenvolvimento e não podem ser ignorados. Um sistema bem testado tem muito mais chance de funcionar corretamente e de atender o que o usuário precisa. Por isso, testar não é só mais uma etapa, mas sim algo essencial pra garantir que o sistema realmente funcione bem.

